

Summary of the Document in Simple Words:

This document outlines two main take-home assignments for an AI/ML engineering role, focusing on building and evaluating AI systems.

Problem 1: Cost-Efficient RAG Application

Goal:

Create a Question Answering (QA) system using a low-cost vector database (like pgvector or Qdrant) to replace expensive managed databases. You need to prove that this approach is cost-effective and performs well in terms of retrieval quality, answer quality, and speed.

What You Need to Do:

- Build a system that can ingest documents (PDF, HTML, MD), split them into chunks, and store them in a vector database.
- Use a model to create embeddings (like text representations) for each chunk and store them with metadata.
- Allow users to ask questions and retrieve the most relevant chunks to generate answers.
- Handle cases where there's no relevant information to avoid making up answers.
- Expose the system as an HTTP endpoint or CLI, with configurable settings and logging.
- Evaluate the system using metrics like recall, precision, and latency.
- Compare the cost of your setup with a managed database for different scales (100K, 1M, 10M vectors).
- Submit a working system, evaluation results, cost comparison, and a README explaining your choices and trade-offs.

What You'll Be Graded On:

- Correctness and how well the system works.
 - Retrieval and answer quality metrics.
 - Cost analysis and assumptions.
 - Code structure, documentation, and clarity.
-

Problem 2: LLM-as-Judge Evaluation Pipeline

Goal:

Build a system that uses another large language model (LLM) to evaluate the quality of AI-generated answers. The system should detect and reduce bias in the evaluation process.

What You Need to Do:

- Create a pipeline that takes test cases (input, model output, expected output, criteria) and uses an LLM to evaluate the answers.
- Use at least one of the following judging methods: pointwise scoring, pairwise comparison, reference-based, or reference-free.
- Define a clear rubric (like correctness, faithfulness, completeness) to evaluate answers.
- Generate reports that compare different configurations (like different prompts or models).

- Log all judge prompts and responses for transparency and replayability.
- Implement bias mitigation techniques, such as testing answer order (A vs B) and controlling for answer length.

What You'll Be Graded On:

- How well you handle bias and implement mitigation strategies.
 - The structure and robustness of your evaluation pipeline.
 - The clarity of your rubric and reporting.
 - The ability to compare different configurations and declare a winner.
-

Summary in One Sentence:

This document asks you to build a cost-effective RAG system and an LLM-based evaluation tool to assess AI-generated answers, with a focus on performance, bias reduction, and transparency.

Summary of the Document in Simple Words:

This document outlines two main take-home assignments for an AI/ML engineering role, focusing on building and evaluating AI systems.

Problem 1: Cost-Efficient RAG Application

Goal:

Create a Question Answering (QA) system using a low-cost vector database (like pgvector or Qdrant) to replace expensive managed databases. You need to prove that this approach is cost-effective and performs well in terms of retrieval quality, answer quality, and speed.

What You Need to Do:

- Build a system that can ingest documents (PDF, HTML, MD), split them into chunks, and store them in a vector database.
- Use a model to create embeddings (like text representations) for each chunk and store them with metadata.
- Allow users to ask questions and retrieve the most relevant chunks to generate answers.
- Handle cases where there's no relevant information to avoid making up answers.
- Expose the system as an HTTP endpoint or CLI, with configurable settings and logging.
- Evaluate the system using metrics like recall, precision, and latency.
- Compare the cost of your setup with a managed database for different scales (100K, 1M, 10M vectors).
- Submit a working system, evaluation results, cost comparison, and a README explaining your choices and trade-offs.

What You'll Be Graded On:

- Correctness and how well the system works.
 - Retrieval and answer quality metrics.
 - Cost analysis and assumptions.
 - Code structure, documentation, and clarity.
-

Problem 2: LLM-as-Judge Evaluation Pipeline

Goal:

Build a system that uses another large language model (LLM) to evaluate the quality of AI-generated answers. The system should detect and reduce bias in the evaluation process.

What You Need to Do:

- Create a pipeline that takes test cases (input, model output, expected output, criteria) and uses an LLM to evaluate the answers.
- Use at least one of the following judging methods: pointwise scoring, pairwise comparison, reference-based, or reference-free.
- Define a clear rubric (like correctness, faithfulness, completeness) to evaluate answers.
- Generate reports that compare different configurations (like different prompts or models).
- Log all judge prompts and responses for transparency and replayability.
- Implement bias mitigation techniques, such as testing answer order (A vs B) and controlling for answer length.

What You'll Be Graded On:

- How well you handle bias and implement mitigation strategies.
- The structure and robustness of your evaluation pipeline.
- The clarity of your rubric and reporting.
- The ability to compare different configurations and declare a winner.

Summary in One Sentence:

This document asks you to build a cost-effective RAG system and an LLM-based evaluation tool to assess AI-generated answers, with a focus on performance, bias reduction, and transparency.